

Rapport de projet

*Conception et réalisation de capteurs
de qualité de l'air*

Projet multidisciplinaire 4A-GP

Tuteur : Yann Rotrou

Max Simonnet
Basile Vidalenc
4A - Génie Physique
Promotion 60

1. Résumé

Ce projet est un projet de conception et de réalisation d'un réseau de capteurs de qualité de l'air intérieur (QAI) pour un établissement scolaire. L'école Saint Paul, dans laquelle prend place notre projet, avait déjà réalisé une session d'auto-évaluation QAI avec des capteurs commerciaux. Elle n'a cependant pas retrouvé dans ces capteurs deux fonctionnalités qui lui sont nécessaires : l'affichage ludique d'un statut instantané et l'acquisition des données de façon permanente. De plus, ces capteurs présentent un coût élevé (120€ pièce pour un capteur NetAtmo).

Ainsi, dans notre projet, chaque capteur, placé dans un boîtier de mesure, mesure la qualité de l'air (CO₂), la température, l'humidité relative et le volume sonore relatif et donne des indications en temps réel et permettant au enseignant.e.s de savoir quand il faut aérer la salle, et aux élèves d'être sensibilisé.e.s. L'acquisition des données se fait ensuite sur une centrale d'acquisition, permettant ainsi des analyses et la mise en place d'actions plus long terme.

2. Remerciements

Tout d'abord, un merci infini à notre tuteur, M. Yann Rotrou. Yann, tu as été une très belle rencontre tant sur le plan humain que sur le plan professionnel cette année. Tu as toujours été à l'écoute et bienveillant lorsque nous étions au point mort. Tu nous as fait confiance en nous laissant une grande autonomie, toujours partant pour chacune de nos idées sans jamais nous pressuriser tout en nous guidant sur les éventuelles difficultés que nous pouvions rencontrer. Un immense merci à toi pour ce projet et cette année !

Ensuite, nous tenions à remercier chaudement M. Jérémie Grisolia, dont l'expertise nous a fait gagner un temps précieux dans la recherche des composants, du protocole de communication adapté et de la bonne interface graphique. Jérémie, la porte de ton bureau était toujours ouverte quand nous avions des questions, des points de blocage, des erreurs... et le tout avec une gentillesse extrême. Merci à toi pour tout cela !

De même, un grand merci à Cathy avec qui nous avons conceptualisé et produit une première ébauche de PCB pour acquérir le signal du capteur. Merci à toi pour ta patience et ton absence de jugement sur nos compétences parfois douteuses en électronique.

Sur le plan organisationnel, nous remercions Lise Arnould, dont l'efficacité et la rapidité extrême nous ont permis d'aller en personne mettre en place le projet au sein même de l'école dans les meilleures conditions.

Un immense merci au Fablab de l'INSA et à ses bénévoles, qui ont passé des heures avec nous pour peaufiner nos design afin de les rendre imprimable en 3D dans les meilleures conditions.

Enfin, merci au CSN et notamment à Louis Chanouha, qui nous a ouvert un sous-réseau au GP.

Merci aussi à tout.es ceux qui ont pu nourrir ce projet en mettant à disposition en ligne des ressources Open Source. La majorité de notre code, notre design 3D, et toutes les connaissances que nous avons acquises : nous les devons à ces personnes. Nous espérons que notre projet pourra un jour inspirer ou aider quelqu'un à son tour.

3. Table des matières

1. Résumé	3
2. Remerciements	3
4. Contexte	5
4.1 Contexte réglementaire	5
4.2 Contexte sanitaire	5
5. Présentation générale du projet	7
5.1 Le Github	7
5.2 Contraintes	7
5.2.1 Au niveau du boîtier de mesures	7
5.2.2 Au niveau du système d'acquisition/gestion	7
5.3 Description et fonctionnement final	8
6. Travail réalisé	10
6.1 Choix des composants	10
6.1.1 Le capteur de CO2 : SCD40	10
6.1.2 Le microcontrôleur : Raspberry Pi Pico W5500-ETH	10
6.1.3 La centrale d'acquisition : Raspberry Pi 4B+	10
6.1.4 L'indication visuelle : les LEDs adressables	11
6.2 La coque 3D	11
6.3 La mise en place réseau	12
6.3.1 Communication MQTT	12
6.3.2 Tests sur l'INSA	13
6.3.3 Mise en place à l'école	13
6.4 Le code de la Pi Pico	13
6.4.1 Nos différentes bibliothèques	14
6.4.2 Le main	15
6.5 L'interface utilisateur	16
6.5.1 Fonctionnement Home Assistant	16
6.5.2 Interface finale	16
7. Bilan	19
7.1 Nos résultat finaux	19
7.2 Retour d'expérience et gestion du projet	19
7.2.1 Organisation	19
7.2.2 Difficulté rencontrés	20
7.2.3 Apprentissages	20
7.3 Perspectives et suites	21
8. Annexes	22
8.1 Scorecard	22

4. Contexte

Ce projet s'inscrit dans notre 4^{ème} année de formation à l'INSA Toulouse dans la spécialité Génie Physique. Nous sommes donc deux étudiants et notre tuteur Yann Rotrou est ingénieur en instrumentation chez Safran. Il est également impliqué dans la vie administrative et associative de l'école Saint Paul de Cesson en tant qu'administrateur OGEC (Organisme de Gestion des Ecoles Catholiques). A ce titre, il siège au Conseil d'Administration de l'école et participe donc aux décisions en terme de d'organisation, de finance et de fonctionnement de l'école. Pour l'école Saint Paul, ce projet fait suite à la campagne d'auto évaluation de 2024, et fait partie du plan d'action présenté dans son rapport. Il repose sur le constat qu'il n'existe pas de capteur commerciaux respectant le besoin d'un affichage ludique, d'une centralisation des données et d'un faible coût, correspondant aux capacités économique d'un établissement scolaire.

4.1 Contexte réglementaire

La conception et l'installation de capteurs de qualité de l'air intérieur dans un établissement scolaire doivent respecter plusieurs normes et recommandations, tant au niveau national qu'international. En France¹, la réglementation encadrant la qualité de l'air dans les établissements recevant du public (ERP) repose sur le Code de la santé publique et des recommandations du Ministère de la Transition écologique et de la Santé, qui définissent notamment des seuils d'exposition² pour les polluants atmosphériques (particules fines PM_{2,5} et PM₁₀, dioxyde de carbone, composés organiques volatils, etc.), ainsi que pour le confort thermique et l'humidité intérieure. Concernant le bruit, les normes françaises (NF S31-080 et NF EN ISO 3382) fixent des limites d'exposition³ dans les espaces publics et scolaires afin de préserver l'audition et le confort des élèves et du personnel.

4.2 Contexte sanitaire

Au niveau international, l'Organisation mondiale de la Santé (OMS) fournit des recommandations visant à protéger la santé des enfants et des adolescents. L'OMS établit des valeurs guides pour les concentrations de polluants atmosphériques dans les espaces intérieurs, notamment :

- Particules fines (PM_{2,5}) : concentration annuelle moyenne inférieure à 5 µg/m³.
- Particules fines (PM₁₀) : concentration annuelle moyenne inférieure à 15 µg/m³.
- Dioxyde de carbone (CO₂) : concentration inférieure à 800 ppm pour garantir une ventilation adéquate et maintenir la concentration et la vigilance.
- Température intérieure : plage recommandée entre 18 et 22 °C pour assurer le confort thermique.
- Humidité relative : plage recommandée entre 30 % et 60 % pour prévenir les risques de moisissures et garantir le confort.

¹Arrêté du 27 décembre 2022 fixant les conditions de réalisation de la mesure à lecture directe de la concentration en dioxyde de carbone dans l'air intérieur au titre de l'évaluation annuelle des moyens d'aération (Légifrance)

²Tableau des normes Qualité de l'Air (ecologie.gouv.fr)

³Arrêté du 25 avril 2003 relatif à la limitation du bruit dans les établissements d'enseignement (Légifrance)

- Niveau sonore : niveau sonore inférieur à 35 dB(A) pour favoriser la concentration et l'apprentissage.

5. Présentation générale du projet

5.1 Le Github

Tout notre projet est disponible en open source sur Github. Vous pouvez y trouver tous les fichiers utilisés, les codes sources et un guide de fabrication et d'utilisation qui permet de refaire chez soi le réseau de capteurs. Tout notre code est donc disponible à cette adresse :

<https://github.com/Maksse/Projet-Multidisciplinaire>

5.2 Contraintes

Pour réaliser ce projet nous nous sommes basé sur un cahier des charge fourni par notre tuteur. Il est résumé ci-dessous les principales contraintes et objectifs présent dans ce cahier.

5.2.1 Au niveau du boîtier de mesures

1. Chaque boîtier ludique doit admettre au minimum un capteur de taux de CO₂ et un capteur de température.
2. Le capteur de taux de CO₂ doit reposer sur la technologie NDIR.
3. L'aspect visuel du boîtier doit être ludique. Il pourra être discuté avec la directrice de l'établissement.
4. Le boîtier doit visuellement présenter un voyant vert lorsque le taux de CO₂ est inférieur à 800ppm, orange lorsque le taux de CO₂ dépasse le 800ppm, et rouge lorsque le taux dépasse 1500ppm. Cela permettra d'ouvrir la fenêtre quand besoin. En optionnel, ces seuils doivent pouvoir être modifiés sur la centrale d'acquisition, pour tous les capteurs.
5. La couleur doit être visible à quelques mètres, pour l'ensemble des élèves de la classe.
6. Les boîtiers doivent être alimentés sur le secteur, ou équipés de batteries ayant une autonomie supérieure à 6 mois.

5.2.2 Au niveau du système d'acquisition/gestion

7. Possibilité de définir des heures de présence (par défaut, identiques pour toutes les pièces, à l'échelle de la semaine. En option : configurable pour chaque pièce).
8. Sur les heures de présence, sauvegarde de toutes les grandeurs physiques mesurées toutes les 10 minutes, et enregistrement centralisé. Le matériel et le protocole de communication sont à discuter⁴.
9. Les boîtiers sont identifiables et on peut leur attribuer sur l'interface un nom facilement modifiable à la journée.
10. Afin de pouvoir maintenir ou remplacer le matériel, toutes les informations nécessaires sont fournies : plan des boîtiers, code source et description de l'installation de l'interface centrale.

⁴après discussions avec notre tuteur et l'école, l'idée d'un système filaire fut privilégiée

5.3 Description et fonctionnement final

Le système final (Figure 1) est donc composé de boîtiers d'acquisitions qui :

- Pilotent des LED qui changent de couleurs selon le niveau de CO₂ (vert < 800ppm ; orange < 1500ppm ; rouge > 1500ppm) en continue.
- Envoyent des données toutes les 10min via Ethernet.

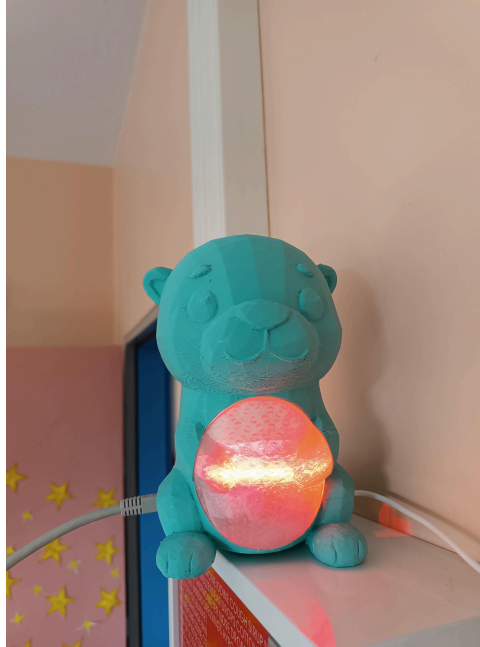


Fig. 1. – Photo de notre loutre en fonctionnement avec un taux de CO₂>1500ppm.

Chaque boîtier de capteur est composé d'une coque imprimée en 3D, d'une Raspberry Pi Pico, d'un capteur Grove Loudness Sensor (en bas de la Figure 2), d'un capteur SCD 40 (petit boîtier en haut de la Figure 2), de LED adressables et de toute la connectique nécessaire (shield Pi Pico, câble Ethernet et alimentation).

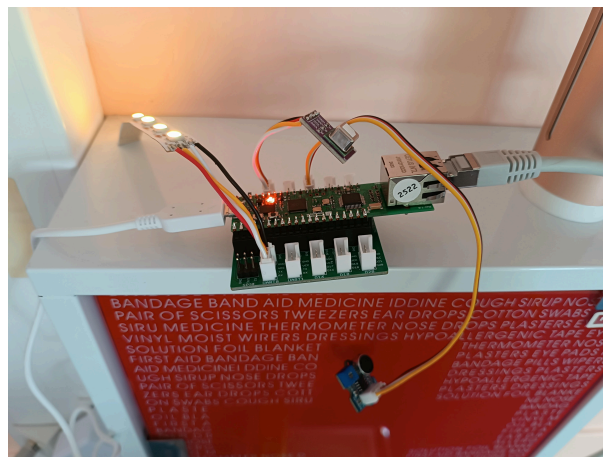


Fig. 2. – Photo du la Pi Pico et de ses composantes, destinés à aller dans la loutre de la Figure 1.

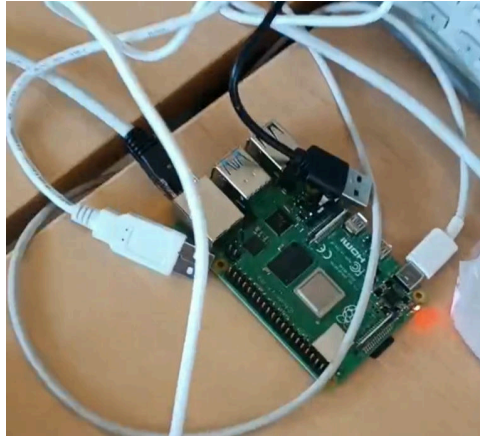


Fig. 3. – Photo de la Raspberry Pi 4, connecté à Ethernet.

Une Raspberry Pi 4 (Figure 3) reçoit ensuite toute la donnée Ethernet et permet d’afficher sur un ordinateur relié au réseau une interface utilisateur Home Assistant qui permet :

- De voir en direct et sur différentes échelles de temps les données pour chaque capteur.
- D’arrêter ou de mettre en marche les capteurs de manière collective ou individuelle, et de manière manuelle ou automatique (en définissant des heures/jours de présence).
- D’effectuer des re-calibrations pour la température et le taux de CO2 pour chaque capteur.
- D’envoyer des rapports automatiques via Telegram.
- De calculer et d’afficher l’indice ICONNE (un indice sur la QAI) pour chaque salle en pouvant choisir l’échelle de temps sur laquelle l’indice est calculé.

Nous allons revenir dans la partie suivante plus en détail sur le fonctionnement global, sur le rôle de chaque composante du projet et sur le travail effectué.

6. Travail réalisé

6.1 Choix des composants

Nous avons choisi les composants en début de projet afin qu'ils correspondent au cahier des charges tout particulièrement sur les aspects :

- ludique (pour le boîtier de mesure qui sera dans les salles)
- d'accessibilité et de reproductibilité (pour que tout un chacun puisse recréer le projet chez lui)
- de conformité aux normes de mesure CO₂ (technologie NDIR)

Pour plus d'informations sur ce qui a motivé nos choix, se référer à la ScoreCard présente en Annexe (Chapitre 8.1).

6.1.1 Le capteur de CO₂ : SCD40

Le SCD40 de chez Sensirion est un capteur de CO₂, Température et Humidité relative. Il utilise la technologie NDIR pour la mesure de CO₂, qui est la technologie fixée par la norme française pour la mesure de CO₂ dans les écoles.

La communication se fait via I2C. Le capteur possède 2 pins d'alimentation (V+ et GND) ainsi qu'une pin SDA pour les données et une pin SCL pour l'horloge. Sensirion a développé une bibliothèque Python que nous avons utilisé pour communiquer avec notre capteur et implémenter les fonctions utiles (plus d'info au Chapitre 6.4.1).

6.1.2 Le microcontrôleur : Raspberry Pi Pico W5500-ETH

Afin d'être parfaitement autonome dans le boîtier de capteur, il est nécessaire de se doter d'un microcontrôleur qui aura pour rôle de communiquer avec les différents modules que nous y branchons : le SCD40 (CO₂), le Loudness Sensor (bruit), les LEDs (indication visuelle) et l'Ethernet.

Pour ce faire, nous avons choisi la Raspberry Pi Pico-W5500-ETH, qui rajoute un module Ethernet fixe à la Pi Pico de base. Ce microcontrôleur dispose des caractéristiques suffisantes à la mise en place de notre projet en terme de mémoire (264kB SRAM, 2MB Flash), de capacité de calcul (133MHz Dual-core) et de communication puisqu'il dispose de Pins I2C, digitaux (I/O) et analogues.

6.1.3 La centrale d'acquisition : Raspberry Pi 4B+

Le choix de la centrale a été fait en même temps que le choix du logiciel pour l'acquisition et le traitement de données, et l'interface graphique. Nous avons décidé de partir sur Home Assistant OS puisque l'école fonctionnait déjà dessus avec une Raspberry Pi 4B+ pour ses capteurs commerciaux. Ainsi, étant déjà lancés dans la suite Raspberry, nous avons décidé de choisir une Pi4B+ également. La Raspberry Pi 4 est compatible avec Home Assistant et possède des caractéristiques suffisantes en terme de stockage (32GB) et de RAM (2GB minimum).

6.1.4 L'indication visuelle : les LEDs adressables

Les LEDs adressables sont un bon outil pour maîtriser l'information visuelle que l'on veut donner. Ces LEDs peuvent être adressées puis contrôlées individuellement via les Pins DT (DATA) et CLK (CLOCK), en étant alimentées via les Pins V+ (tension d'alimentation) et GND (masse) (plus d'info au Chapitre 6.4.1).

6.2 La coque 3D

Pour réaliser la forme du boîtier de capteur qui sera présent dans les salles de classe, nous nous sommes basés sur ce modèle⁵ de loutre, créé par AR3DP.



Fig. 4. – Modèle initial du boîtier (créateur : AR3DP)

Ce modèle est sous licence **CC BY-NC 4.0** qui nous permet de le modifier et de le partager gratuitement, en citant l'auteur.

A l'aide de l'outil *Meshmixer*, nous avons creusé l'intérieur de la loutre pour ensuite extruder l'avant et l'arrière sur *Fusion 360*. Le trou à l'arrière sert à placer le capteur dans la loutre, et celui à l'avant à placer un ventre transparent qui permet de diffuser la lumière produite par les LEDs. Enfin, 2 trous rectangulaires ont été placés sur les côtés de la loutre afin de faire passer les câbles USB et Ethernet, nécessaires respectivement à l'alimentation et à la connexion au réseau. L'impression en fabrication additive a été faite au FabLab de l'INSA, en PETG bleu cyan pour la loutre et en PETG transparent pour le ventre.

⁵<https://www.thingiverse.com/thing:7198225>

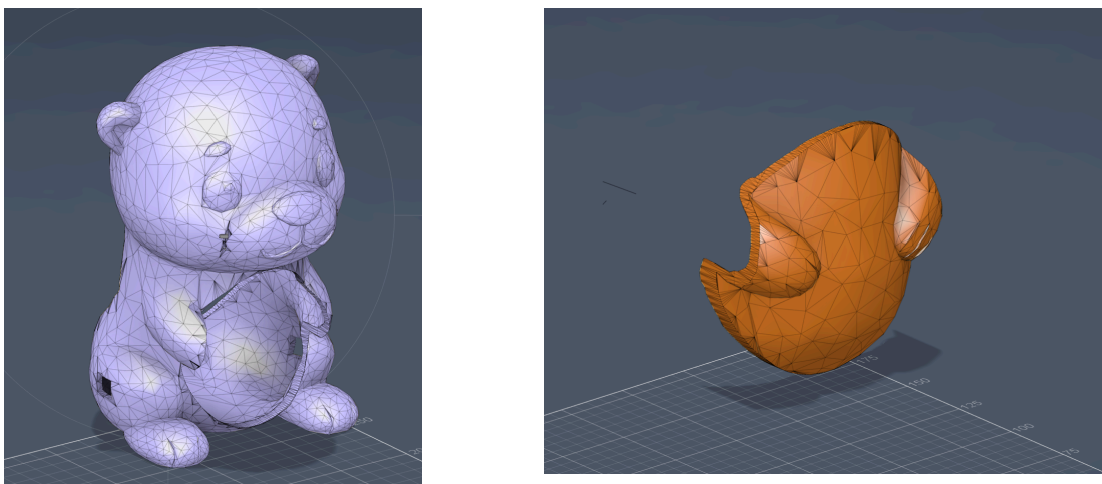


Fig. 5. – Loutre et ventre, fichier prêt pour impression (.stl)

6.3 La mise en place réseau

6.3.1 Communication MQTT

Pour assurer la communication entre les boîtiers de capteurs et la centrale d'acquisition, nous avons choisi le protocole MQTT (Message Queuing Telemetry Transport). Ce protocole repose sur un modèle « *publish/subscribe* » : chaque Pi Pico publie ses données sur un « *topic* » (un canal de communication identifié par une chaîne de caractères), et la Raspberry Pi 4 qui fait tourner Home Assistant s'abonne à ces topics pour récupérer les données en temps réel. Un broker Mosquitto, installé sur la Pi 4, joue le rôle d'intermédiaire central et redistribue chaque message à tous les abonnés concernés.

Nous avons fait le choix d'implémenter nous-mêmes la couche MQTT dans notre bibliothèque `lib_mqtt.py` plutôt que d'en utiliser une existante, afin de garder un code léger et adapté aux contraintes mémoire de la Pi Pico. Concrètement, à chaque démarrage, la Pi Pico établit une connexion TCP avec le broker en lui envoyant un paquet CONNECT contenant son identifiant, ses identifiants de connexion et un testament (`will_topic()`). Ce dernier permet au broker de publier automatiquement un message offline si le capteur venait à se déconnecter brutalement, ce qui permet à Home Assistant de détecter une panne.

Une fois connectée, la Pi Pico fonctionne dans les deux sens. D'un côté, elle publie toutes les 10 minutes ses mesures (CO_2 , température, humidité, bruit) au format JSON sur son topic dédié. De l'autre, elle est abonnée à un topic de commande `"/set_status"` sur lequel Home Assistant peut envoyer des ordres : allumer ou éteindre le système, modifier l'offset de température, changer les seuils d'alerte CO_2 , ou encore forcer une recalibration du SCD40. Ces messages entrants sont traités par la fonction `reception_message()` du `main`, qui est une fonction de rappel (callback) et s'exécute automatiquement dès qu'un message arrive. En effet, `check_msg()` ne déclenche automatiquement `reception_message()` que lorsque qu'elle détecte un message, afin de ne jamais bloquer le programme.

6.3.2 Tests sur l'INSA

Avec l'aide du CSN, nous avons mis en place un sous-réseau de test en salle d'instrumentation 2.15. Ce sous-réseau nous a permis de mimer un réseau local régit par une box, ce qui se rapproche le plus de l'architecture réseau de l'école (peu de hiérarchie réseau/sous-réseau, ce qui facilite la communication entre ports). Ce sous-réseau contenait 4 ports Ethernet reliés entre eux et permettant de brancher :

- 1 Raspberry Pi 4 (centrale d'acquisition/Execution de Home Assistant),
- 1 PC (accès à Home Assistant)
- 2 Raspberry Pi Pico (mesure/indication visuelle/envoi de données à la centrale)

A l'aide de ce sous-réseau nous avons pu :

- créer l'interface utilisateur sur Home Assistant et d'implémenter des codes `.yaml` correspondant aux exigences de l'école via le PC (voir Chapitre 6.5.1)
- vérifier la robustesse de notre couche MQTT (créée par un code python) : envoi et réception de message sur la Pi Pico et sur Home Assistant.
- identifier les points critiques qui pourraient poser problème lors de la mise en place à l'école :
 - l'IP du Broker MQTT (i.e. de la Pi 4), essentielle à connaître et à indiquer dans le `main.py` des Pico afin de pouvoir communiquer
 - l'IP de la passerelle, du masque de sous-réseau et du DNS, à indiquer dans `lib_mqtt.py`

6.3.3 Mise en place à l'école

Lors de la mise en place du réseau, basée sur la « Feuille de route mise en place du réseau » présente sur notre GitHub, nous avons suivi les étapes suivantes afin de tester la communication MQTT :

- récupération des IPs utiles (passerelle, masque de sous-réseau, DNS) et implémentation dans notre code
- fixation de l'IP de la Pi 4 (donc du Broker MQTT) et implémentation dans notre code
- connexion au Home Assistant via un PC connecté au réseau
- test de tous les ports Ethernets de l'école et vérification qu'ils sont sur le même réseau
- test final en faisant tourner notre code

Mis à part un problème de temps d'attente de connexion au réseau pour les Pi Pico dans certaines salles (voir Chapitre 7.2.2), la mise en place s'est déroulée sans accroc.

6.4 Le code de la Pi Pico

Nous allons dans cette section aborder le code implémenté sur la Raspberry Pi Pico que nous avons créé et testé sur l'environnement VS Code. Ce code permet 2 choses :

- De fonctionner de manière autonome et d'indiquer grâce à la bande LED le taux de CO2.
- De recevoir et d'envoyer des messages via MQTT.

Nous n'allons pas aborder tous les codes réalisés, il y a notamment en plus sur le Github des codes de tests des différents composants et du réseau Ethernet. Dans cette section nous allons dans une première partie revenir sur les différentes librairies que nous avons créées, puis sur le code principal dans une seconde partie.

6.4.1 Nos différentes bibliothèques

- `lib_led.py` :

Cette bibliothèque se base sur une bibliothèque récupérée depuis un Github de pilotage python des LED WS2812B. On y retrouve la fonction `config_led()` qui permet d'initialiser les LEDs en passant en argument le nombre de LED, l'intensité voulue et les pins sur lesquelles elles sont connectées. La fonction `indicateur_visuel()` qui permet de changer la couleur du bandeau LED en passant en argument le taux actuel de CO2. La dernière fonction est `piloter_led()` qui permet d'allumer complètement (lumière blanche) ou d'éteindre le bandeau LED.

- `lib_son.py` :

Cette librairie permet d'utiliser le Grouve Loudness Sensor.

On y trouve la fonction `config_port_son()` qui initialise le capteur en passant en argument la pin sur laquelle il est connecté. La deuxième fonction est `niveau_sonore()` qui renvoie une valeur brute du niveau sonore comprise entre 0 et 65535.

- `lib_scd4x.py` :

Cette bibliothèque se base sur une bibliothèque récupérée depuis le Github officiel du SCD40 ainsi que sur une bibliothèque Adafruit. Elle permet de contrôler le SCD40.

On y retrouve la fonction `config_port_scd()` qui permet de définir les pins SDA et SCL. La fonction `obtenir_donnees()` qui renvoie le taux de CO2, la température et l'humidité relative. Enfin deux fonctions permettent de modifier les paramètres de fonctionnement du SCD40. `change_parametres()` permet de mettre à jour l'offset de température (que l'on doit corriger pour prendre en compte l'échauffement du capteur) et l'altitude de fonctionnement. `recalibration()` permet de forcer une calibration sur un niveau de CO2 connu. Sinon le SCD40 effectue une calibration automatique toutes les semaines en se basant sur les valeurs les plus basses rencontrées.

- `lib_mqtt.py` :

Cette bibliothèque contient la `class MQTTClient` que l'on doit initialiser en donnant les informations de notre broker (ici relatives à la Pi 4).

On y retrouve la fonction `connecter_ethernet()` qui permet de configurer la connexion Ethernet (SPI et matériel). La fonction `subscribe()` permet de s'abonner à un sujet MQTT. La fonction `check_msg()` permet de vérifier si un message a été publié sur le sujet MQTT et `publier()` permet d'envoyer un message sur un sujet MQTT.

- `lib_IP_MAC.py` :

Cette librairie contient la fonction `connect_ethernet()` qui met à jour la configuration (SPI et matériel) et renvoie l'adresse IP et MAC de la Pi Pico qui exécute le code. Elle est utile car la communication MQTT nécessite de connaître ces adresses. Cette fonction permet donc d'automatiser la récupération de ces adresses et d'utiliser le `main.py` avec n'importe quelle Pi Pico.

6.4.2 Le main

Nous allons maintenant aborder le fonctionnement global du main, en revenant sur ses différentes fonctions, puis sur son corps.

- `reception_message()` :

Cette fonction de rappel s'exécute automatiquement dès qu'un message MQTT est reçu sur un sujet auquel la Pi Pico est abonnée. Elle décode le message pour intercepter les ordres provenant de Home Assistant. Elle gère l'allumage ou la mise en pause du système (ordres **ON** et **OFF**) via la variable globale `global systeme_actif`, et permet également de modifier dynamiquement à distance l'offset de température, les seuils d'alerte CO2 ou de forcer une recalibration du capteur SCD40.

- `tache_leds_et_capteur()` :

Cette tâche asynchrone s'exécute en continu toutes les 10 minutes. Son rôle principal est de « nourrir » le chien de garde (*Watchdog Timer*) pour prouver que le système n'est pas bloqué, puis de récupérer les mesures physiques (taux de CO₂, température, humidité et niveau sonore) afin de mettre à jour le dictionnaire global `donnees_actuelles`. Si le système est actif, elle ajuste en temps réel la couleur du bandeau LED en fonction des seuils de CO2 définis. Si le système est mis en pause, elle s'assure de l'extinction physique des LEDs.

- `tache_mqtt()` :

Cette seconde tâche asynchrone gère la communication réseau avec le broker MQTT. Elle configure d'abord la fonction de réception des messages, puis s'assure en boucle de l'état de la connexion. En cas de déconnexion, elle reconnecte le client, s'abonne au sujet de commande `"/set_status"` et publie son état de disponibilité (`"online"`). Lorsque la connexion est établie, elle vérifie l'arrivée de nouveaux messages et publie régulièrement (toutes les 10 minutes par défaut) les données des capteurs au format `.json`, à condition que le système soit actif.

- `main()` :

Il s'agit de la fonction principale et du point d'entrée du programme. Elle commence par initialiser la sécurité du système via le chien de garde `wdt()` configuré sur un délai de 8 secondes. Elle configure ensuite l'ensemble du matériel (bandeau LED, capteur sonore et capteur SCD4x avec une réinitialisation forcée de sa mesure périodique). Après avoir initialisé la connexion Ethernet et le client MQTT, elle orchestre et lance simultanément les deux tâches asynchrones (`tache_leds_et_capteur()` et `tache_mqtt()`) grâce au moteur `uasyncio.gather`.

6.5 L'interface utilisateur

6.5.1 Fonctionnement Home Assistant

Nous avons dans ce projet configuré notre Home Assistant sur une Raspberry Pi 4B+ de l'INSA. Nous avons ensuite reconfiguré en environnement similaire à l'école Saint Paul. L'école disposait déjà d'une Raspberry 4 avec HA installé, nous avons donc dû configurer la connexion MQTT. Puis en copiant les codes sources de notre environnement INSA sur celui de l'école. Ces différents fichiers où est le code sont :

- **configuration.yaml :**

Ce fichier contient tout les capteurs qui vont permettre de recevoir et de traiter les données envoyées par la Pi Pico. Chaque capteur est associé à un « sujet », qui est un canal MQTT. Quand HA reçoit les données en `.json`, il les reçoit pour un sujet et il peut donc associer à un indicateur. Ce fichier contient également tous les éléments qui seront présents sur le tableau de bord. Ce sont « les entités », qui sont des objets avec une identité et des attributs comme un nom, une unité et des valeurs min et max par exemple.

- **Le tableau de bord :**

Le tableau de bord est l'endroit où sont encodés tous les attributs de nos indicateurs (taille, couleur, titre, ...). Pour chaque indicateur, on associe une entité, l'indicateur avec le nom « CO2 salle 1 » sera donc connecté au taux de CO2 réel qui a pour entité « CO2_salle_1 » reçu sur le sujet de conversation MQTT « topic_CO2_salle_1 ».

- **automation.yaml :**

Ce fichier contient toutes les automatisations qui sont des actions effectuées automatiquement sous certaines conditions. La principale automation de notre interface est d'allumer et d'éteindre tous les capteurs selon des heures de fonctionnement définies, dans notre cas donc d'allumer tout les capteurs entre 8h et 21h.

- **script.yaml :**

Le fichier script lui permet aussi d'effectuer d'autres types d'actions, par exemple des actions MQTT. Pour nous, le fichier script contient des scripts pour envoyer certains types de messages MQTT.

Une fois tous ces codes collés dans le Home Assistant de l'école et après vérification que la compilation a réussi, nous avons pu effectuer nos premiers tests.

6.5.2 Interface finale

La Figure 6 et 7 sont des captures d'écran de notre tableau de bord, qui est donc l'espace depuis lequel la gestionnaire de l'école va utiliser les données des capteurs.

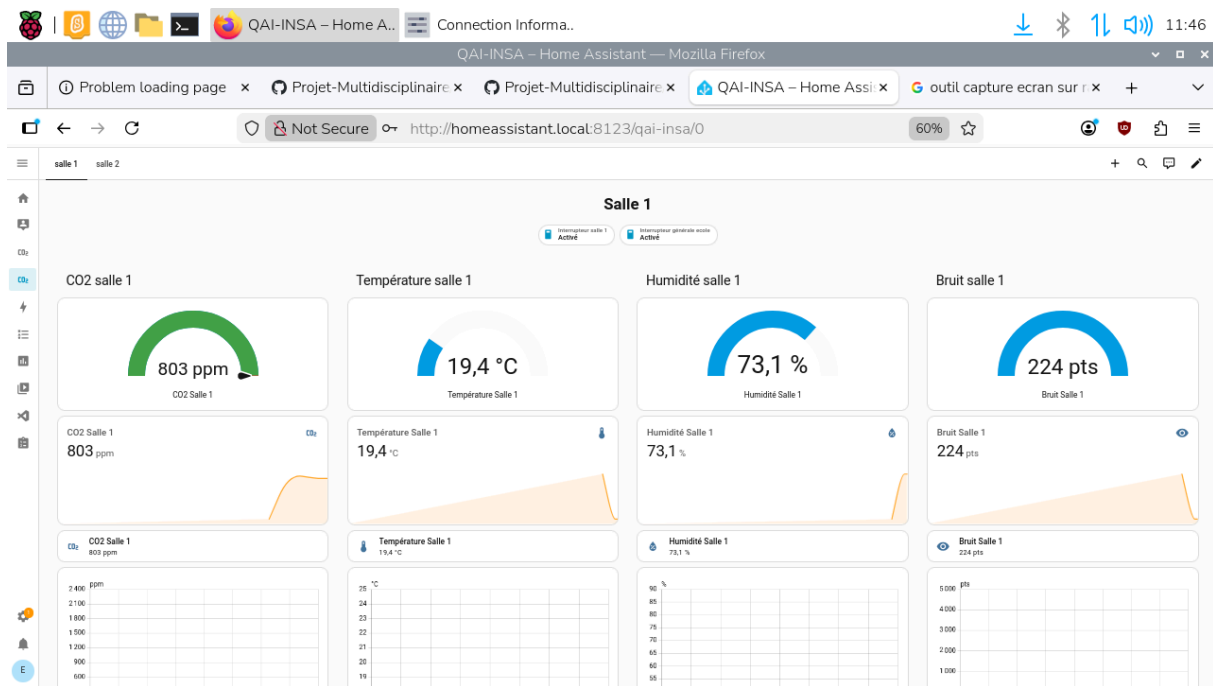


Fig. 6. – Capture d'écran du tableau de bord des capteurs.

Nous allons revenir sur les différents éléments qui composent ce tableau de bord et ce qu'il est possible de faire :

- **Les salles :**

En haut de la Figure 6, on voit qu'il est possible de choisir la salle que l'on veut visualiser. Toute l'interface est donc accessible pour chaque salle, donc pour chaque boîtier. Au moment où nous avons réalisé l'installation à l'école, deux capteurs étaient présents.

- **Les indicateurs :**

Sur la Figure 6 on peut voir les différents indicateurs disponibles pour chaque grandeur. On a notamment la jauge en temps réel, l'évolution à court terme, les statistiques de long terme et l'état (disponible ou indisponible).

- **Les interrupteurs :**

En haut de la Figure 6 on peut voir des interrupteurs. Il y a pour chaque salle un interrupteur spécifique. L'interrupteur interrompt l'envoi de la donnée et l'allumage des LEDs. Il y a aussi un interrupteur global qui contrôle en même temps tous les interrupteurs, et qui permet donc d'éteindre/allumer toutes les salles.

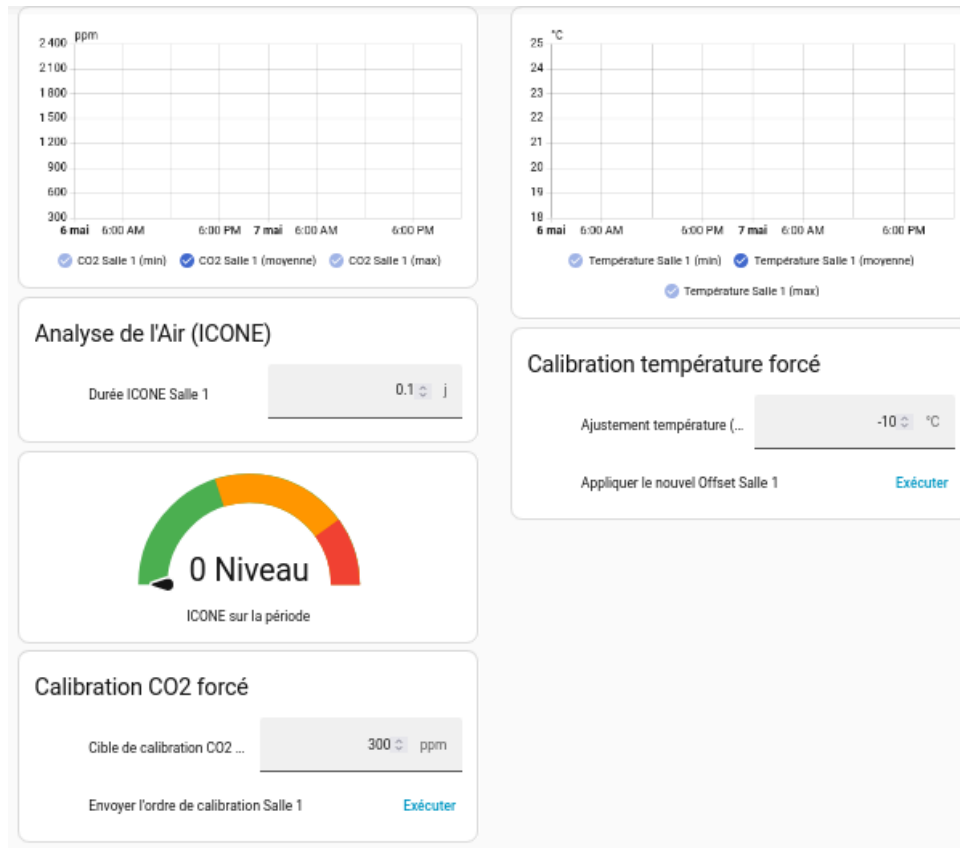


Fig. 7. – Capture d'écran du tableau de bord des capteurs (en dessous de ce qui est visible sur la Figure 5).

• L'indice ICONE :

L'indice ICONE est un indicateur réglementaire utilisé pour évaluer le niveau de confinement de l'air intérieur dans les écoles et les lieux publics en général. Il varie de 0 (confinement nul) à 5 (confinement extrême) comme ceci :

$$N = 8.3 \cdot \log_{10}(1 + f_1 + 3f_2)$$

Où :

- N est la valeur brute de l'indice (arrondie à l'entier le plus proche pour obtenir la note ICONE finale de 0 à 5), plus il est faible, plus la QAI est bonne.
- f_1 représente la fraction de temps où la concentration en CO_2 est comprise entre 800 ppm et 1500 ppm.
- f_2 représente la fraction de temps où la concentration en CO_2 dépasse 1500 ppm.

• Les commandes :

Parmi les commandes qui sont visibles sur la figure on a d'abord le temps sur lequel sera calculé l'indice ICONE. Ce qui permet aussi bien de calculer une note sur la dernière heure que le dernier mois. On a également une commande pour forcer la calibration du SCD40 à un certain seuil de CO_2 connu. On a finalement une commande pour appliquer un offset de température au SCD40 (la mesure de la température peut dériver à cause de l'échauffement dû au fonctionnement). Toutes ces commandes fonctionnent sur le principe d'envoi de messages MQTT.

7. Bilan

7.1 Nos résultat finaux

Au final nous avons pu livrer à l'école Saint Paul un réseau de 2 capteurs fonctionnels avec une interface utilisateur facile à prendre en main. L'école pourrait avoir besoin d'un réseau allant jusqu'à 17 capteurs mais le but est aussi de jauger les besoins avant d'en fabriquer de nouveaux.

Le point positif de ce réseau est que le coût unitaire d'un capteur est moins cher que dans le commerce (< 80euro) pour une norme similaire (technologie NDIR), une plus grande réparabilité (changement d'un composant contre changement de tout le capteur, voir Chapitre 8.1). Un autre avantage est la plus grande flexibilité et le nombre important de fonctions :

- Il y a un capteur de bruit.
- Tout le projet est modifiable sous condition de se plonger dans le code HA et python.
- Il y a un aspect « paramétrage pédagogique » : dans d'autres établissements, les capteurs tout le temps en alarme, et les enseignant.es et élèves ont finit par les ignorer. Nos capteur peuvent changer de seuil d'alarme.
- Il y a une centralisation des données pour prendre des actions plus long terme (consignes générales d'ouvertures des fenêtres, ouverture permanente d'au moins une porte, dimensionnement de la VMC,...)
- Le projet à un aspect ludique et les capteurs plaisent aux enfants.

Le principal point de vigilance est que le réseau est utilisable par des non-initiés en informatique mais en cas de panne, bugs ou si l'on veut modifier quelque chose, cela nécessite des bases en minimales en informatique.

7.2 Retour d'expérience et gestion du projet

7.2.1 Organisation

Pour mener à bien ce projet nous avons dû nous organiser. Nous avons d'abord fait des visio-conférences régulières avec notre tuteur, toutes les deux semaines environ. Durant ces visios nous le tenions au courant de l'avancement du projet, nous discussions et validions les choix à faire (choix des composants ou du langage de code ou design du capteur par exemple) et nous lui posions également des questions, principalement sur l'environnement de l'école, et parfois technique (sur le fonctionnement de HA par exemple). Nous avons également communiqué par WhatsApp pour les informations plus en continue et l'organisation des visios et des déplacements. Concernant les déplacements, nous avons vu Yann une première fois à Toulouse en milieu d'année puis nous avons effectué un déplacement à l'école Saint Paul début mai pour mettre en place le réseau. Nous avons pour ce déplacement préparé en amont une feuille de route avec un emploi du temps de la journée, un tableau avec le matériel nécessaire et une liste des problèmes potentiels. Ce déplacement s'est très bien déroulé, nous avons pu découvrir l'école, mettre en place le réseau. Nous avons également pu expliquer et aider la gestionnaire et Yann à prendre l'outil en main, de sorte qu'ils soient le plus possible autonomes.

7.2.2 Difficulté rencontrés

Une première difficulté a été rencontrée au début du projet. Il nous a semblé cohérent financièrement et pédagogiquement pour nous de concevoir les PCBs qui permettraient de communiquer avec le SCD40. Avec l'aide de Catherine Crouzet nous avons designé et produit un exemplaire qui n'a malheureusement pas marché après une batterie de tests (notamment réalisés avec Jérémie Grisolia). Nous avons donc finalement commandé notre SCD40 directement sur PCB, mais cela nous a permis d'aborder certains savoirs-faire en micro-soudure et en KiCad avant les cours du second semestre.

Une autre difficulté est arrivée lors de la mise en place du réseau à l'école. En effet, nous avons fait face à des problèmes réseau et nous avons debugger notre code pendant 2 à 3 heures, ligne par ligne. Il nous manquait à certains endroits de notre code des temps d'attente. Cela nous a ralenti et mis en retard sur le planning de la journée mais ne nous a pas empêché de finir l'installation de nos capteurs.

Finalement ce projet nécessitait en grande partie des connaissances sur des sujets que nous n'avions jamais abordés en cours. Notamment le réseau Ethernet, la conception 3D et l'instrumentation sur micro-contrôleur. Nous comptons cela comme une difficulté dans le sens où prendre en main certaines notions nous a demandé du temps. La mise en place d'un sous-réseau au GP nous a par exemple demandé beaucoup plus de temps que prévu car nous découvrions les contraintes d'Ethernet au fur et à mesure et nous avons dû faire de nombreux aller-retour de mails avec le CSN.

7.2.3 Apprentissages

Ce projet nous a beaucoup appris. D'abord sur le plan théorique. Nous avons découvert de nombreux domaines, notions, fonctions python et beaucoup de choses sur la suite Raspberry. Nous sommes satisfaits de ces apprentissages dans le sens où nous sentons qu'ils pourront être ré-utilisés dans d'autres contextes. La suite Raspberry est en effet utilisée dans des domaines très variés, le code python aussi, et les réseaux Ethernet également.

Sur le plan de l'organisation et de la communication nous avons aussi mené ce projet dans une autonomie assez grande, tout en communiquant avec des acteurs assez variés. Que ce soit à l'INSA, avec l'école et notre tuteur ou entre nous.

Et c'est peut être ici que ce projet nous a le plus appris : à travailler à plusieurs. Souvent des après-midi entières, nous avons fonctionné toujours avec l'autre, toujours dans la communication, l'entraide et le soutien. Ce projet aura été une grande source d'apprentissage sur le plan humain et relationnel. Nous pouvons noter la progression que nous avons réalisée au niveau de la gestion du stress, de l'organisation de planning à deux sur une semaine/un mois/un an, le partage de connaissances ou encore le partage des tâches.

7.3 Perspectives et suites

Pour finir ce projet il nous reste encore certaines choses à faire. Nous devons finaliser le Github et le manuel de fabrication. Aussi nous devons probablement assurer un travail de suivi avec l'école au cas où le réseau rencontrerait des problèmes complexes.

Pour ce qui est des perspectives futures, nous espérons que ce projet pourra servir à d'autres, pour tout ou partie. Notamment aux écoles car nous avons vu au cours de ce projet qu'il y a une réelle problématique de qualité d'air en classe. Ce projet propose en effet un outil pour repenser la qualité de l'air lors de l'enseignement. Nous pouvons penser aussi peut être aux salles de sport, aux lieux accueillant du public ou même à l'INSA !

8. Annexes

8.1 Scorecard

ScoreCard — Selection des composants

Reseau de capteurs CO2 — Justification des choix techniques

1. Avantage

- Inconvenient

✓ Composant retenu

Capteur CO2	SCD40 Sensirion	Unit CO2 SCD40 GoTronic	SCD30 Sensirion
	RETENU + Mesure NDIR precise (± 50 ppm) + Compensation humidité/temperature intégrée + Interface I2C et bibliothèques déjà développées + Très abordable (~19€) - Nécessite de l'équipement électronique (soudures) - Plage de mesure un peu juste (400-2000pm)	+ Même caractéristiques que le SCD40 + Intégration très facile (connexions déjà soudées) - Trop cher (43€)	+ Grande plage de mesure (400-10000) + Meilleure précision (~30ppm) - Dimensions plus grandes (potentiels problèmes pour le boîtier) - 6 Pins de connexion - moins bien intégrable sur Raspberry Pi Pico
Microcontrôleur	Pi Pico-W5500-ETH Raspberry	ESP32 Espressif	Arduino Uno Arduino
	RETENU + Ethernet intégré + Programmation en MicroPython + Bibliothèques intuitives et très nombreuses + Communication I2C, UART, Analogique - Soudures nécessaires - Prise Ethernet à proximité	+ Wi-Fi + Bluetooth intégrés + Puissant (dual-core 240 MHz) + Ecosystème riche - Environnement extrêmement lourd sur PC (~5GB) - Plus complexe à programmer qu'en MicroPython	+ Tres simple a prendre en main + Large communauté et contribution OpenSource - Dimensions trop grandes - Pas d'Ethernet natif (module complémentaire à acheter)

Protocole réseau	MQTT	
	RETENU + Extrêmement léger et simple d'utilisation + Infrastructure existante + Broker Mosquitto libre - Nécessite une IP fixe pour la centrale d'acquisition	
Interface utilisateur	Home Assistant OS Home Assistant RETENU + Déjà installé à l'école (personnel déjà formé et habitué) + Simplicité d'utilisation et ergonomique + Communauté active + Compatible avec MQTT - Nécessite une IP fixe pour la centrale d'acquisition	Nodered ESP32 natif + Communauté active + Système de noeuds facilement adaptable aux projets IoT - Visuellement moins ergonomique pour le personnel de l'école
Alimenta- tion	Batterie externe 5V 2000 mAh + Adapte aux usages extérieurs + Transportable + Recharge facile - Coût trop élevé pour un réseau de plusieurs capteurs - Autonomie trop faible (12h max.) donc réduction de la praticité	Alimentation secteur 5V USB RETENU + Simple et fiable + Pas de gestion de batterie - Nécessite une prise secteur à proximité - Non adapte en extérieur

Comparaison avec un équivalent commercial

Projet Multi — Capteurs NetAtmo

1. Avantage

- Inconvénient

✓ Composant retenu

Boîtier	Projet Multi Max/Basile	Capteurs NetAtmo NetAtmo
	RETENU + Couleur d'impression au choix + Design mignon et ludique - Fragile aux chocs - Temps d'impression	+ Design très propre - Non ludique et seulement 2 couleurs de boîtier au choix
Indication visuelle	Projet Multi Max/Basile	Capteurs NetAtmo NetAtmo
	RETENU + Code couleur clair (vert/orange/rouge) + Seuils CO2/Bruit et couleurs adaptables selon la volonté des utilisateur.ice.s - Soudures nécessaires	+ Ready-to-use - Code couleur peu clair - Pas de modifications possibles
Réparabilité/ Adaptabilité	Projet Multi Max/Basile	Capteurs NetAtmo NetAtmo
	RETENU + Tous les composants sont remplaçables et ne coûtent pas plus de 20€ pièce + Adaptable grâce au code python modifiable + Modules supplémentaires ajoutables (Pins libres sur la Pico) - Fer à souder nécessaire - Connaissances en réseau/code/électronique	- Rachat complet nécessaire (120€) - aucune adaptabilité (boîte noire)